



Digital Decibel Sound Level Indicator with Peak-Hold

A register-level design in VHDL

Computer Organization & Architecture (BTS30101)

Sayantan Bharati | BWU/BTS/25/503 | Section B

B.Tech CSE, Batch 2025 | Faculty: Dr. Subhankar Shome

Brainware University | 03-11-2026



Where the Design Starts

The analog world ends at one 8-bit bus



Assume

The ADC hands the design one 8-bit unsigned sample per clock.

Study

Entity, registers, comparator, synchronous reset, testbench.

Skip

Microphone, amplifier and ADC circuit. Not part of this design.



The Interface: Entity

Five ports define the whole design

```
entity sound_meter is
  Port (
    clk  : in  STD_LOGIC;
    rst  : in  STD_LOGIC;
    din  : in  STD_LOGIC_VECTOR(7 downto 0);
    db   : out STD_LOGIC_VECTOR(7 downto 0);
    peak : out STD_LOGIC_VECTOR(7 downto 0)
  );
end sound_meter;
```

clk

in
1 bit

rst

in
1 bit

din

in
8 bit

db

out
8 bit

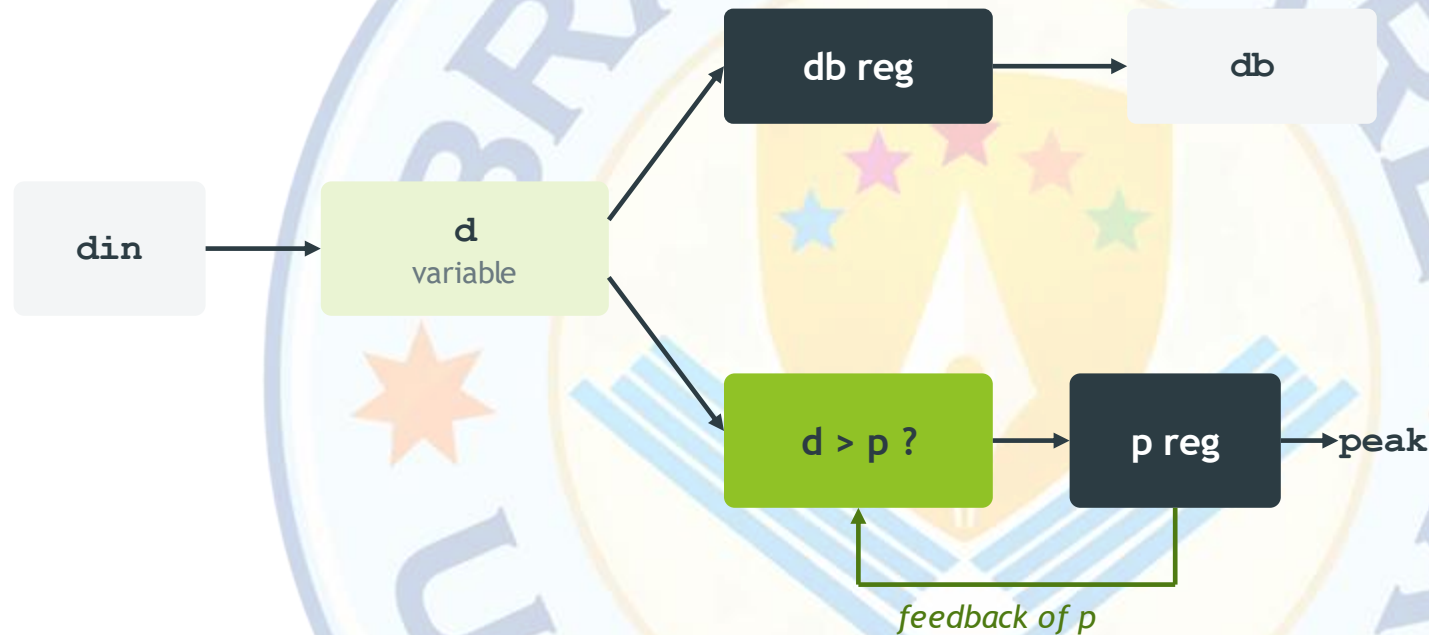
peak

out
8 bit

8-bit data width: values 0 to 255.

Datapath View

Two registers and one comparator



No arithmetic beyond one compare. The peak is simply the largest value *p* has stored so far.

Clocked Process and Reset

Everything happens on the rising clock edge

```
process(clk)
  variable d : unsigned(7 downto 0);
begin
  if rising_edge(clk) then
    if rst = '1' then
      db  <= (others => '0');
      peak <= (others => '0');
      p   <= (others => '0');
    else
      -- datapath (next slide)
    end if;
  end if;
end process;
```

Edge-triggered

rising_edge(clk): flip-flops.

Synchronous

rst is read only at a clock edge.

Reset state

db, peak, p all become 0.

Peak-Hold: Compare and Store

The core of the design in three lines

```
d := unsigned(din);  
db <= std_logic_vector(d);  
  
if d > p then  
    p <= d;  
end if;  
  
peak <= std_logic_vector(p);
```

d > p

New maximum. Store it in p.

d <= p

No branch taken. p keeps its old value.

p never decreases, except through reset. That is what makes it a hold.



Variable vs Signal

Why peak trails by one clock

variable d :=

Updates immediately. Used for db and the compare in the same edge.

signal p <=

Updates after the edge. peak reads the old p.

Testbench instant: din changes from 10 to 30

	edge at 125 ns	edge at 135 ns
din	30	30
p	30	30
peak	10	30

peak = p delayed one clock. db = din delayed one clock.



Testbench Structure

Instantiate, clock, reset, stimulate

```
 uut: entity work.sound_meter
   port map (clk => clk, rst => rst,
             din => din, db => db,
             peak => peak);
```

```
 clk <= not clk after 5 ns;
```

```
 rst <= '1'; wait for 20 ns;
```

```
 rst <= '0';
```

```
 din <= x"10"; wait for 100 ns;
```

```
 din <= x"30"; wait for 100 ns;
```

```
 -- ... x"A0", x"20", x"00"
```

Clock

10 ns period, toggles every 5 ns.

Reset

20 ns at start, 30 ns mid-run.

Stimulus

10 values, 100 ns each.

Simulation Results

Settled value at the end of each 100 ns step

din	10	30	50	70	90	60	40	A0	20	00	R	80
peak	10	30	50	70	90	90	90	A0	A0	A0	00	80

R = rst asserted. Green cells show the peak changing.

Ignored

60 and 40 arrive after
90. peak stays at 90.

New max

A0 beats 90. peak
moves up, then 20 and
00 are ignored.

Reset

rst clears peak to 00.
Then 80 starts a new
hold.



COA Concepts and Next Steps

What this small design demonstrates

In this design

- Registers (p, db)
- Comparator and feedback
- Clocked sequential logic
- Synchronous reset

Not yet in the code

- Log (dB) conversion: db is the raw sample
- RMS windowing
- Hold timer or decay

Thank you.

Refs: Patterson & Hennessy; Stallings; Hamacher
Complete Code(Github): [Click here](#)